

CognitiveOS - Technical PRD

Generated: December 8, 2025

Complexity: High

Estimated Duration: 10-14 semaines (3 phases)

1. Executive Summary

CognitiveOS est un système de mémoire neuro-symbolique local-first pour LLM, combinant un Knowledge Graph vectorisé, un agent "Hippocampe" gérant la curation de mémoire, et un processus de consolidation asynchrone inspiré du sommeil humain. L'objectif est de créer une infrastructure de mémoire persistante, privée et portable qui s'interface avec n'importe quel modèle de langage.

2. Project Requirements

Functional Requirements

- [FR-1] Extraction automatique d'entités et relations depuis les conversations (via Function Calling)
- [FR-2] Stockage dans un Knowledge Graph hybride (structure + embeddings vectoriels)
- [FR-3] Retrieval contextuel avec injection du sous-graphe pertinent dans le prompt
- [FR-4] Agent Hippocampe gérant les opérations CRUD sur le graphe (Create, Update, Delete)
- [FR-5] Processus de consolidation asynchrone (clustering, abstraction, pruning)
- [FR-6] Versioning temporel des faits (gestion des contradictions et évolutions)
- [FR-7] Interface de visualisation du graphe de mémoire
- [FR-8] Export/Import du graphe pour portabilité

Non-Functional Requirements

- Performance:** Retrieval < 100ms pour un graphe de 10K nœuds
- Scalabilité:** Support jusqu'à 100K nœuds/500K edges par utilisateur
- Privacy:** Données 100% locales, chiffrement au repos optionnel
- Compatibilité:** Python 3.10+, macOS/Windows/Linux

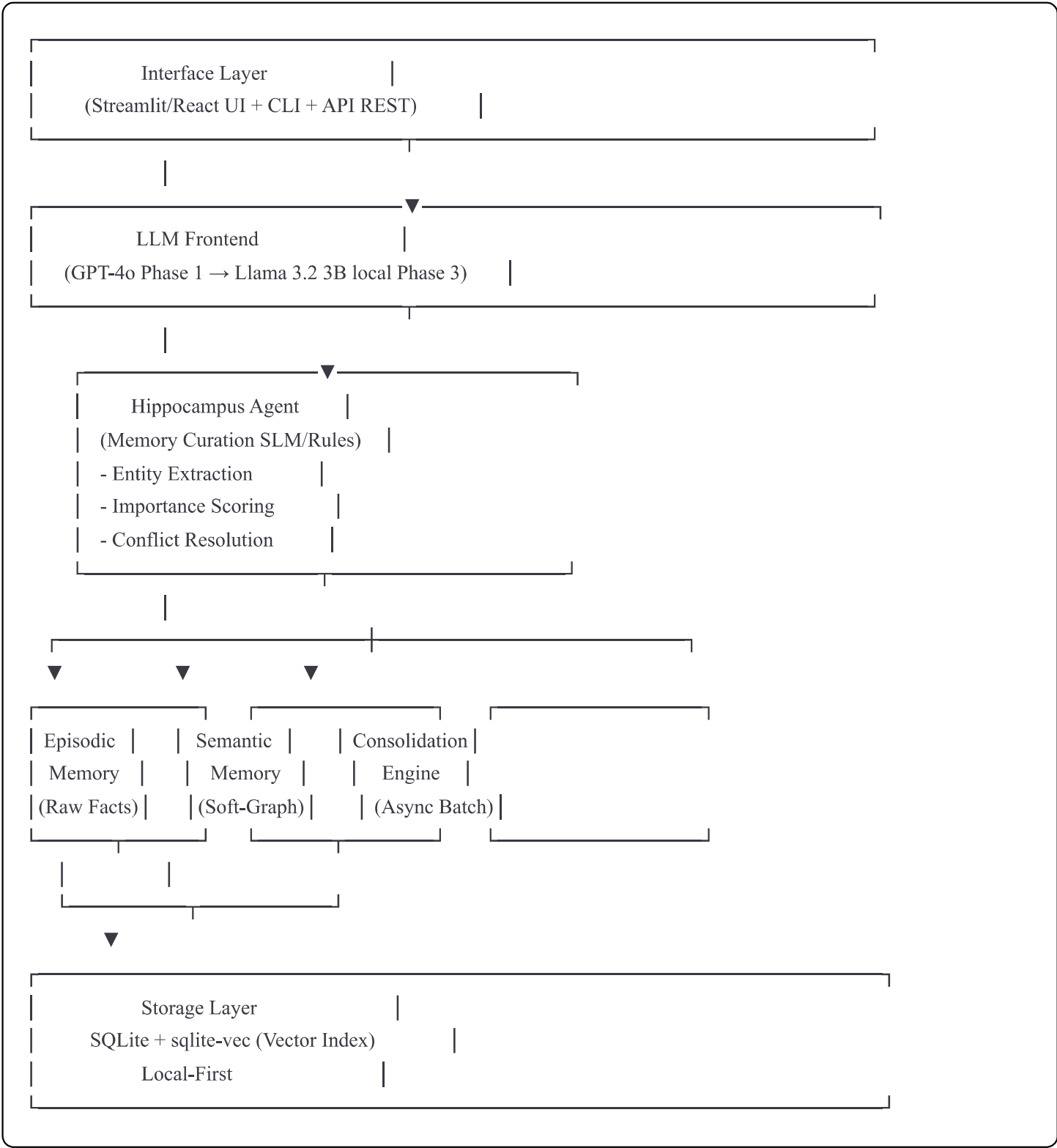
Constraints

- Timeline:** ~3 mois pour MVP fonctionnel

- **Budget:** Open-source, coûts API LLM pour Phase 1 uniquement (~\$20-50)
- **Team:** 1 développeur fullstack
- **Hardware cible:** Laptop standard (8GB RAM minimum), GPU optionnel pour Phase 3

3. Architecture Système

Vue d'ensemble



Modèle de données du Soft-Graph

python

```
# Node Structure
{
  "id": "uuid",
  "type": "Person|Concept|Event|Preference|Skill|...",
  "label": "Pizza",
  "embedding": [0.12, -0.45, ...], # 384-dim
  "properties": {
    "created_at": "2025-01-15T10:30:00Z",
    "confidence": 0.85,
    "activation_count": 3,
    "last_activated": "2025-01-20T14:00:00Z"
  }
}

# Edge Structure
{
  "id": "uuid",
  "source": "node_uuid_1",
  "target": "node_uuid_2",
  "relation": "LIKES|KNOWS|WORKS_AT|HAS_EVIDENCE|...",
  "embedding": [0.08, 0.32, ...], # Pour relations nuancées
  "properties": {
    "status": "active|archived",
    "valid_from": "2024-01-01",
    "valid_to": null,
    "weight": 0.9,
    "context": "mentioned during discussion about food"
  }
}
```

4. Recommended Stack

Overview

Layer	Technology	Version	Justification
Orchestration	LangGraph	0.2.x	Framework agent stateful, 11.7k stars, 4.2M downloads/mois
LLM (Phase 1)	OpenAI GPT-4o	-	Qualité extraction, Function Calling natif
LLM (Phase 3)	Llama 3.2 3B	via Ollama	Local, 128K contexte, 2GB stockage
Graph (Phase 1)	NetworkX	3.4+	50M+ downloads, prototypage rapide

Layer	Technology	Version	Justification
Graph (Phase 2+)	SQLite + schema relationnel	3.45+	Portable, performant, pas de serveur
Vector Search	sqlite-vec	0.1.x	Successeur de sqlite-vss, pure C, cross-platform
Embeddings	sentence-transformers	3.x	all-MiniLM-L6-v2, 384 dims, rapide
UI	Streamlit	1.40+	Prototypage rapide, visualisation
UI (optionnel)	React + D3.js	18.x	Visualisation graphe interactive

LangGraph (Orchestration)

Chosen: LangGraph

- GitHub: 11.7k stars, commits actifs (décembre 2025)
- PyPI: 4.2M downloads/mois (tendance ↑)
- **Why chosen:** Framework officiel LangChain pour agents stateful. Support natif de la mémoire court/long terme, checkpointing, human-in-the-loop. Parfait pour orchestrer le flux Conversation → Hippocampus → Graph.

NetworkX (Phase 1 - Prototype)

Chosen: NetworkX

- GitHub: 15k+ stars, très actif
- PyPI: 50M+ downloads (avril 2024)
- **Why chosen:** Standard de facto pour graphes en Python. API simple, sérialisation JSON native. Limitations connues sur la mémoire pour grands graphes → acceptable pour Phase 1 (< 5K nœuds).

SQLite + sqlite-vec (Phase 2+)

Chosen: SQLite avec sqlite-vec pour vector search

- sqlite-vec: Successeur actif de sqlite-vss (abandonné)
- Pure C, pas de dépendances, fonctionne partout (y compris WASM)
- Support int8/float32/binary vectors avec metadata filtering
- **Why chosen:** Local-first, zero config, performant. Un seul fichier `(.db)` = portabilité maximale.

Sentence Transformers (Embeddings)

Chosen: all-MiniLM-L6-v2

- 384 dimensions, bon compromis qualité/vitesse
- ~80ms par embedding sur CPU

- **Why chosen:** Standard pour embeddings légers, intégration LangChain native.

Additional Libraries

Purpose	Library	Stats	Notes
Async processing	APScheduler	18k stars	Jobs de consolidation
Config	Pydantic	22k stars	Validation settings
Serialization	orjson	6k stars	JSON rapide pour graphe
CLI	Typer	16k stars	Interface ligne de commande
Testing	pytest	12k stars	Tests unitaires/intégration

5. Alternatives Considered

KùzuDB (Graph Database)

- **Pros:** Embedded, Cypher natif, 18x plus rapide que Neo4j, vector search intégré
- **Cons:** **Projet archivé en 2025** (critical), écosystème réduit
- **Why rejected:** Risque de maintenabilité trop élevé malgré les performances. SQLite + schema relationnel offre plus de pérennité.

Neo4j

- **Pros:** Mature, Cypher, écosystème riche
- **Cons:** Serveur requis, overhead pour usage local, Community Edition limitée
- **Why rejected:** Contradictoire avec l'approche local-first/embedded.

ChromaDB / Pinecone

- **Pros:** Optimisés pour vector search, API simple
- **Cons:** Pas de structure graphe native, ChromaDB = qualité variable, Pinecone = cloud only
- **Why rejected:** Besoin de structure relationnelle du Knowledge Graph, pas juste de vector store.

CrewAI (Agent Framework)

- **Pros:** 30k stars, multi-agent natif
- **Cons:** Plus orienté "équipes d'agents" que workflow de mémoire
- **Why rejected:** LangGraph plus adapté pour un agent unique avec état complexe.

6. Risks & Limitations

Technical Risks

Risk	Probability	Impact	Mitigation
Extraction d'entités inconsistante	Medium	High	Prompt engineering itératif, validation schema JSON
Latence retrieval avec graphe large	Medium	Medium	Index vectoriel + caching prédictif
Qualité du modèle local (Phase 3)	Medium	High	Fine-tuning optionnel, fallback vers API
Conflicts mémoire épisodique/sémantique	Low	Medium	Versioning temporel strict
sqlite-vec breaking changes	Low	Medium	Pin version, tests de régression

Known Limitations

- **Cold Start RL (Phase 3):** L'agent Hippocampe nécessite un dataset de bootstrap. Mitigation: génération synthétique via GPT-4 puis SFT.
- **Scalabilité au-delà de 100K nœuds:** Non testé. Pour ce volume, migration vers DuckDB + extension graph recommandée.
- **Multi-utilisateur:** Architecture single-user. Multi-user nécessiterait isolation des DB et auth.

Dependencies & Blockers

- Disponibilité API OpenAI (Phase 1 uniquement)
- Stabilité sqlite-vec (projet relativement récent)
- Hardware minimum pour Phase 3 (8GB RAM, idéalement NPU/GPU)

7. Estimations

Time Breakdown

Phase	Duration	Notes
Phase 1: Dirty Prototype	2 semaines	Boucle de base fonctionnelle
Phase 1.5: Stress Test	3-4 jours	200-300 interactions synthétiques
Phase 2: Persistence + Consolidation	4-5 semaines	SQLite, UI, batch process
Phase 3: Local & Private	3-4 semaines	Ollama, optimisation, polish
Buffer & Documentation	1 semaine	
Total	10-14 semaines	

Complexity Factors

- (+) Extraction d'entités = problème NLP semi-résolu via LLM

- (+) NetworkX → SQLite migration = refactoring significatif
- (+) UI de visualisation graphe = complexité frontend
- (-) Pas d'auth/multi-user = simplification
- (-) Local-first = pas d'infra cloud

Assumptions

- Développeur à temps partiel (~15-20h/semaine)
 - Accès API OpenAI pour Phase 1
 - Machine avec 8GB+ RAM disponible
 - Familiarité avec Python, async, et concepts ML de base
-

8. User Stories

Epic 1: Core Memory Loop

- **US-1.1:** As a user, I want to have a conversation with the LLM and have relevant facts automatically extracted so that I don't have to manually tag information.
 - Acceptance criteria: Entities/relations extracted avec >80% precision sur dataset de test
 - Priority: **Must**
- **US-1.2:** As a user, I want the LLM to recall information from past conversations so that I don't have to repeat myself.
 - Acceptance criteria: Retrieval correct dans >70% des cas où info pertinente existe
 - Priority: **Must**
- **US-1.3:** As a user, I want to see what the system remembers about me so that I can verify and correct information.
 - Acceptance criteria: UI affiche graphe navigable avec nœuds/edges
 - Priority: **Should**

Epic 2: Memory Management

- **US-2.1:** As a user, I want outdated information to be automatically archived so that the system stays current.
 - Acceptance criteria: Conflits détectés et marqués "archived" avec historique
 - Priority: **Must**

- **US-2.2:** As a user, I want trivial information to be forgotten over time so that the system doesn't get cluttered.
 - Acceptance criteria: Pruning automatique des nœuds non-activés >30 jours
 - Priority: **Should**
- **US-2.3:** As a user, I want repeated patterns to be abstracted into general knowledge so that retrieval is efficient.
 - Acceptance criteria: Clustering crée nœuds parents avec HAS_EVIDENCE links
 - Priority: **Should**

Epic 3: Privacy & Portability

- **US-3.1:** As a user, I want all my data stored locally so that my memories are private.
 - Acceptance criteria: Zero network calls sauf API LLM (Phase 1)
 - Priority: **Must**
- **US-3.2:** As a user, I want to export my memory graph so that I can back it up or migrate.
 - Acceptance criteria: Export JSON complet, importable sur nouvelle instance
 - Priority: **Could**
- **US-3.3:** As a user, I want to run the system entirely offline so that I'm not dependent on external services.
 - Acceptance criteria: Phase 3 fonctionne avec Ollama sans internet
 - Priority: **Should**

9. Task Breakdown

Legend

-  Blocker (blocks other tasks)
-  High Priority
-  Normal Priority
-  Nice-to-have

Phase 1: Dirty Prototype (2 semaines)

ID	Task	Estimate	Dependencies	Priority
1.1	Setup projet Python (pyproject.toml, structure folders)	2h	-	
1.2	Config Pydantic pour API keys, settings	2h	1.1	

ID	Task	Estimate	Dependencies	Priority
1.3	Implémenter wrapper OpenAI avec Function Calling	4h	1.2	
1.4	Définir schema JSON pour opérations mémoire (CREATE_NODE, CREATE_EDGE, etc.)	3h	1.3	
1.5	Implémenter classe MemoryGraph avec NetworkX	4h	1.4	
1.6	Implémenter méthodes CRUD sur MemoryGraph	4h	1.5	
1.7	Créer prompt système pour extraction d'entités	3h	1.4	
1.8	Implémenter retrieval: query → vecteur → nœuds voisins	4h	1.5	
1.9	Intégrer sentence-transformers pour embeddings	2h	1.5	
1.10	Créer boucle conversationnelle principale	3h	1.7, 1.8	
1.11	Sérialisation/désérialisation graphe JSON	2h	1.5	
1.12	CLI basique pour tester (Typer)	2h	1.10	
1.13	Tests unitaires pour MemoryGraph	3h	1.6	
1.14	Test manuel: 10 messages, vérifier recall	2h	1.10	

Subtotal Phase 1: ~40h

Phase 1.5: Stress Test (3-4 jours)

ID	Task	Estimate	Dependencies	Priority
1.5.1	Script génération dialogues synthétiques (GPT-4)	3h	1.10	
1.5.2	Générer 200-300 interactions variées	2h	1.5.1	
1.5.3	Exécuter stress test, collecter métriques	2h	1.5.2	
1.5.4	Identifier edge cases (doublons, contradictions, orphelins)	3h	1.5.3	
1.5.5	Documenter problèmes et solutions	2h	1.5.4	

Subtotal Phase 1.5: ~12h

Phase 2: Persistence & Consolidation (4-5 semaines)

ID	Task	Estimate	Dependencies	Priority
2.1	Design schema SQLite pour graphe (nodes, edges, properties)	4h	Phase 1	
2.2	Implémenter SQLiteGraph remplaçant NetworkX	8h	2.1	
2.3	Intégrer sqlite-vec pour index vectoriel	6h	2.2	
2.4	Migration helper NetworkX → SQLite	3h	2.2	
2.5	Implémenter retrieval hybride (vector + graph traversal)	6h	2.3	

ID	Task	Estimate	Dependencies	Priority
2.6	Optimiser queries avec index appropriés	4h	2.5	
2.7	Implémenter versioning temporel (edge properties)	4h	2.2	
2.8	Créer ConsolidationEngine (classe de base)	4h	2.2	
2.9	Implémenter détection de patterns répétitifs	6h	2.8	
2.10	Implémenter clustering → abstraction hiérarchique	8h	2.9	
2.11	Implémenter pruning (forgetting curve)	4h	2.8	
2.12	Implémenter résolution de conflits temporels	4h	2.7	
2.13	Scheduler APScheduler pour batch consolidation	3h	2.8	
2.14	Setup Streamlit app de base	3h	2.2	
2.15	Vue conversation avec mémoire injectée	4h	2.14	
2.16	Visualisation graphe interactive (PyVis ou D3)	8h	2.14	
2.17	Panneau stats (nœuds, edges, activations)	3h	2.14	
2.18	Tests d'intégration SQLite + vector search	4h	2.5	
2.19	Benchmark latence retrieval (target < 100ms)	3h	2.18	
2.20	Documentation API interne	4h	2.5	

Subtotal Phase 2: ~93h

Phase 3: Local & Private (3-4 semaines)

ID	Task	Estimate	Dependencies	Priority
3.1	Setup Ollama + Llama 3.2 3B	2h	Phase 2	
3.2	Adapter wrapper LLM pour Ollama API	4h	3.1	
3.3	Tester extraction d'entités avec modèle local	4h	3.2	
3.4	Ajuster prompts pour Llama 3.2 (si nécessaire)	6h	3.3	
3.5	Implémenter Hippocampus Agent local (règles ou SLM)	8h	3.2	
3.6	Générer dataset de training pour Hippocampus (via GPT-4)	6h	Phase 2	
3.7	Fine-tuning optionnel Llama pour curation (LoRA)	12h	3.6	
3.8	Predictive caching du sous-graphe	6h	2.5	
3.9	Optimisation mémoire (lazy loading, pagination)	4h	3.1	
3.10	Mode offline complet (no network)	3h	3.2	
3.11	Export/Import fonctionnel (JSON + embeddings)	4h	2.2	
3.12	Chiffrement optionnel de la DB (SQLCipher)	4h	2.2	
3.13	Tests end-to-end en mode local	4h	3.10	
3.14	Benchmark performance modèle local vs API	3h	3.13	

ID	Task	Estimate	Dependencies	Priority
3.15	README complet + guide d'installation	4h	3.13	
3.16	Docker image optionnelle	4h	3.15	

Subtotal Phase 3: ~78h

Summary

- **Total tasks:** 50
- **Total estimated time:** ~223h (~14-15 semaines à 15h/semaine)
- **Critical path:** 1.1 → 1.3 → 1.4 → 1.5 → 1.10 → 2.1 → 2.2 → 2.3 → 2.5 → 3.1 → 3.2 → 3.3

10. Success Metrics

Phase 1 (Prototype)

- ☐ Boucle conversation → extraction → recall fonctionnelle
- ☐ Graphe persiste entre sessions (JSON)
- ☐ Recall correct sur 7/10 questions nécessitant mémoire

Phase 2 (Persistence)

- ☐ Migration complète vers SQLite
- ☐ Latence retrieval < 100ms sur 10K nœuds
- ☐ Consolidation batch fonctionnelle (clustering + pruning)
- ☐ UI visualisation utilisable

Phase 3 (Local)

- ☐ Fonctionne 100% offline avec Ollama
- ☐ Performance extraction acceptable (>60% precision vs GPT-4)
- ☐ Temps de réponse < 5s pour génération complète

Appendix: Research Data

Key Findings

sqlite-vss vs sqlite-vec:
sqlite-vss n'est plus maintenu activement. L'auteur (Alex Garcia) recommande sqlite-vec comme successeur.

sqlite-vec est en pure C, sans dépendances, et supporte metadata filtering.

KùzuDB Archival:

Le projet KùzuDB a été archivé. Les releases existantes restent utilisables mais sans nouvelles features ni bugfixes. Migration recommandée vers alternatives.

LangGraph Adoption:

Utilisé en production par Klarna (85M users), AppFolio, Elastic. Support natif pour mémoire court/long terme depuis LangChain v1.0.

Llama 3.2 3B Performance:

- Storage: 2GB (quantized)
- VRAM: 3.7GB
- Context: 128K tokens
- Outperforms Gemma 2 2.6B et Phi 3.5-mini sur instruction following et summarization

Sources

- <https://github.com/langchain-ai/langgraph>
- <https://github.com/asg017/sqlite-vec>
- <https://github.com/kuzudb/kuzu> (archived)
- <https://ollama.com/library/llama3.2>
- <https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/>
- <https://networkx.org/>
- <https://simonwillison.net/2024/Sep/25/llama-32/>

Ce PRD est un document vivant. Itérer sur les estimations après chaque phase.